Reviewer Pack — Minimal Symplectic Form Rank and Resolution Proof Width Ineq…

Entry f85f16ebd8a3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 20:02:18 UTC

Minimal Symplectic Form Rank and Resolution Proof Width Inequality

Entry ID: `f85f16ebd8a3` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-03 20:02:18 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry (Form Theory) **Field B** (complexity object): Resolution Proof Complexity

Statement:

> For every Boolean satisfiability problem (SAT) instance φ, the minimal rank of its associated symplectic form ωφ is linearly related to its resolution proof width w(φ), such that min_rank(ωφ) ≥ c * w(φ), where c is a constant.

Rationale (proposer's reasoning):

> Symplectic forms encode geometric and dynamical properties of spaces, which may provide insight into the complexity of computational processes. The minimal rank of a symplectic form could reflect the complexity of resolving logical queries in φ, suggesting a connection between geometric structures and computational hardness.

Taxonomy category: `SymplecticFormRankResolutionProofWidth` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ad7389d4f0f886d2`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For the conjecture that minimal symplectic form rank is linearly related to resolution proof width, we will consider an instance φ supported if min_rank(ωφ) ≥ c * w(φ) for all seeds and a condition falsified if any seed has min_rank(ωφ) < c * w(φ).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `min_rank symplectic form AND resolution proof complexity` - `symplectic geometry AND linear relationship resolution proof width` - `resolution proof width inequality AND associated symplectic form rank`

Top relevant hits considered: - [http://arxiv.org/abs/1002.3099v3] Tamed Symplectic forms and SKT metrics - [http://arxiv.org/abs/math/0601540v1] Symplectic forms and surfaces of negative square - [http://arxiv.org/abs/1811.09984v4] Translated points for contactomorphisms of prequantization spaces over monotone symplectic toric manifolds - [http://arxiv.org/abs/1703.01731v2] On the existence of infinitely many non-contractible periodic orbits of Hamiltonian diffeomorphisms of closed symplectic - [http://arxiv.org/abs/1902.01261v2] On a local systolic inequality for odd-symplectic forms - [http://arxiv.org/abs/math/0611768v5] The Invariant Symplectic Action and Decay for Vortices - [http://arxiv.org/abs/2509.16734v1] Multigenerational Inequality - [s2:10.5194/essd-2020-44-ac2] Replies to comments of reviewer 2

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_phi(n):
    phi = []
    for _ in range(n):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(3)]
        if all(lit not in phi[-1] and -lit not in phi[-1] for lit in clause):
            phi.append(clause)
    return phi

def symplectic_form(phi):
    m = len(phi)
    n = len(phi[0])
    omega = [[0] * (n + 2) for _ in range(n + 2)]
    for i in range(m):
        for j in range(n):
            omega[j][j+1] += phi[i][j]
            omega[j+1][j] -= phi[i][j]
    return omega

def min_rank(omega):
    m = len(omega)
```

```
    n = len(omega[0])
    rank = 0
    for i in range(n):
        if any(omega[j][i] != 0 for j in range(m)):
            rank += 1
    return rank

def resolution_width(phi):
    stack = []
    for clause in phi:
        stack.append(clause)
    while stack:
        clause = stack.pop()
        new_clause = None
        for lit in clause:
            if -lit in clause:
                return len(stack) + 1
            for other_clause in stack:
                if any(-lit == x or lit == x for x in other_clause):
                    new_clause = [x for x in other_clause if x != -lit and x != lit]
                    break
            if new_clause is not None:
                break
        if new_clause is not None:
            stack.append(new_clause)
    return len(stack) + 1

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    min_ranks = []
    widths = []
    for n in n_values:
        phi = generate_phi(n)
        omega = symplectic_form(phi)
        min_rank_val = min_rank(omega)
        width = resolution_width(phi)
        min_ranks.append(min_rank_val)
        widths.append(width)

    mean_min_rank = sum(min_ranks) / len(min_ranks)
    mean_width = sum(widths) / len(widths)
    c = mean_min_rank / mean_width
    conjecture_holds = all(m >= c * w for m, w in zip(min_ranks, widths))
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "min_rank_over_width",
        "metric_value": c,
        "instances_tested": len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
```

```python
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_min_ranks = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_min_ranks} std=0.0 support_fraction={support_fractior
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_398cde46.py", line 98, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_398cde46.py", line 71, in run_trial
    phi = generate_phi(n)
          ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_398cde46.py", line 22, in generate_phi
    if all(lit not in phi[-1] and -lit not in phi[-1] for lit in clause):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_398cde46.py", line 22, in <genexpr>
    if all(lit not in phi[-1] and -lit not in phi[-1] for lit in clause):
                      ~~~^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means we cannot verify whether the conjecture holds or not. | next: Re-run the test with proper error handling to ensure it completes without crashing and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	26018
2	preregistration	ollama_remote	glm4:latest	0	0	9890
3	novelty	ollama_remote	glm4:latest	0	0	9237
4	novelty	ollama_remote	glm4:latest	0	0	21367
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26049
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22089
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32580
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10945
9	judge	ollama_remote	glm4:latest	0	0	12285

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 170460 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/f85f16ebd8a3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f85f16ebd8a3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/f85f16ebd8a3.tar.gz` (if generated)